

Step 0 Results — Layer-3 Probe of the Denodo Data Catalog (dev)

Findings, schema implications, and next steps for the DSI backend (Phase 1, read-only)

Denodo → DSI backend, tri-layer model

Denodo-DSI Backend Progress (Phase 1 – Read Only):

- Step 0: Layer-3 probe on dev (before locking the schema)
- Step 1: Data contract: tri-layer model → DSI tables
- Step 2: API client layer (separate from the DSI backend)
(The goal of Step 2 is to build DenodoAPIClient - a standalone module that concentrates **all HTTP communication with Denodo** in one place, so the DSI backend never has to know HTTP exists.)
- Step 3: Building denodo.py backend (read-only) based on Kayla's ndp.py
- Step 4 – Testing
- Step 5 - Documentation and delivery

DSI Contributor Docs and Denodo Data Catalog REST API Docs

I researched the DSI contributor docs and Denodo Data Catalog REST API docs. Two findings anchor everything:

- DSI already formalizes "read-only" constraint: backends are loaded as either BACK-READ or BACK-WRITE (the distinction lives in core.py), and write functions like 'ingest_artifacts' can only be called on a BACK-WRITE backend. So Phase 1 = implement the BACK-READ surface only.
- To be compatible with DSI core middleware, backends must interface with Python built-in data structures and the 'collection' library, and any contributed backend must include unit tests in 'backends/tests' — untested pull requests are not accepted. That defines the acceptance criteria.
- On the Denodo side: the API supports HTTP Basic, stateless OAuth (enabled via 'oauth.statelessEnabled=true'), and stateful OAuth (JSESSIONID + X-XSRF-TOKEN) — you've already mapped all of these in your notebook, so Phase 1 only needs an 'AuthProvider' that abstracts over these modes.

For Unit testing: DSI's contributor rules require unit tests in backends/tests, and tests can't hit the real network. But our OAuthManager *opens a browser popup* to authenticate — impossible in CI. With a common interface, we can write a FakeAuthProvider (see the sketch below) that injects a dummy header, and the entire backend becomes testable without real OAuth.

```
class FakeAuthProvider(AuthProvider):
    def apply(self, headers):
        headers["Authorization"] = "Bearer test-token"
        return headers
```

Also relevant for shaping the read API on DSI: the existing Sqlite/DuckDB backends define the de-facto backend interface — ‘process_artifacts’ (read everything into a nested OrderedDict keyed by table name), ‘query_artifacts’, ‘get_table’, ‘list’, ‘summary’, ‘display’, and the find family (find_table, find_column, find_cell, find_relation) returning ValueObjects. Mirroring this vocabulary in our file, let’s say denodo.py will make other people’s review and merge much easier.

The ideal Tri-layer Model:

DSI table	Layer	Key columns
denodo_databases	1 (VDB)	db_name, description, server_id
denodo_views	2 (dataset)	view_name, db_name, view-level metadata, property values (HTML-stripped)
denodo_columns	2.5 (schema)	view_name, db_name, column_name, type, description
denodo_resources	3 (sparse)	<u>view_name</u> , <u>db_name</u> , <u>resource_type</u> , <u>uri</u> , <u>source_field</u>

1. Motivation and Purpose of Step 0

The tri-layer model we have intended to build — databases/VDBs, views, and additional files or resources — left one open design question: the third layer. Maxen said that attachments and documentation could sometimes be linked externally or referenced in metadata fields, so it’s worth planning to capture those as a third layer for completeness. That meant the denodo_resources table could not be responsibly designed on paper; we needed to measure what layer-3 actually looks like in the wild. Step 0 was therefore a empirical probe with a single deliverable sentence: "Of the 15,014 views on dev, N have layer-3 resources, distributed across these metadata fields." Running it before Step 1 will tell us the shape of the denodo_resources or what isn’t a layer-3 resource, which reshapes the other tables.

Executive Summary

Step 0 is complete. We probed all 15,007 active views in the dataportal database on the dev catalog (99.9% coverage) to answer one question before locking the Step 1 data contract: how many views actually carry layer-3 resources, and where do they live? The answer is that layer-3 is real but sparse and simple. Only 829 views (about 5.5%) carry any resource-type signal, and only 544 carry a genuine documentation URL — always exactly one per view, and 86% of those point to Oracle table-reference documentation. Two fields we had flagged as possible resource containers turned out, on verification, to be something else entirely: source-system lineage and access-request workflow pointers. These findings

directly determine the shape of the `denodo_resources` layer and let us defend every field-level decision in the Step 1 schema with data rather than assumption.

2. Method

A resumable Jupyter probe (`denodo_layer3_probe_oauth.ipynb`) authenticated via our existing OAuth Authorization Code flow and pulled the paginated list of all views in dataportal, then fetched view-details for each one keyed on the (`viewName`, `databaseName`) pair. Property values — including `RICH_TEXT` fields — were HTML-stripped, URLs were extracted from `href` attributes before stripping, and every hit was written to SQLite (`probe_results.db`) with checkpoints every 500 views so the sweep could resume after interruption. After the sweep, targeted verification queries against the SQLite database classified each candidate field by inspecting its actual values and URL domains.

3. Results

The headline funnel is below. Each number in it backs a specific schema decision, discussed in the next section.

Count	Population	Share of probed views
15,007	Active views probed on dev (99.9% coverage of dataportal)	100%
5,366	Views with any metadata signal (URLs, links, or person references)	35.8%
829	Views with resource-type signals	5.5%
544	Views with a genuine documentation URL (exactly one per view)	3.6%

The verification queries then classified the candidate fields as follows.

Metadata field	What verification showed	Classification for the schema
<code>description</code>	544 URLs, exactly 1 per view; 86% Oracle table-reference documentation (<code>etrm.live</code> , <code>docs.oracle.com</code>)	Genuine Layer-3 resource → <code>documentation_url</code> , a nullable scalar on the view
<code>ODS Details / ODS DB Link</code>	Oracle database-link identifiers (<code>EBS_LINK</code> 63%, <code>SNFLWR_LINK</code> , <code>ARIBA_LINK</code>); 0 of 1,347 values contain a URL; 182 literally say "NOLINK"	Layer-1 lineage metadata → <code>source_system</code> field, not a resource

Details / Access Role Request	93% of URLs point to accessit.lanl.gov, the LANL access-request portal	Workflow pointer → access_instructions field, not a document
Primary POC / Data Owner	Person links, the dominant signal type across all views	Contacts, modeled separately from document resources

The sweep also surfaced three operational facts worth recording. The dev and prod APIs drift on field names (the list endpoint returns db on dev but databaseName on prod), which the Step 2 must normalize. 15,007 view-details calls keyed on (viewName, databaseName) succeeded with 99.9% coverage, confirming the decision to key on the name pair rather than on view IDs, which are known to contain duplicates.

4. What This Means for the Step 1 Schema

First, the shape of denodo_resources simplifies dramatically. Because documentation URLs appear at exactly one per view, the honest model on dev is a nullable scalar documentation_url column on the view rather than a one-to-many resources table.

Second, the reclassifications keep the resource layer clean. ODS DB Link values are Oracle database-link identifiers, not documents, so they belong in a source_system lineage field at layer 1. Access Role Request URLs are pointers into the access-request workflow, so they become access_instructions. Without Step 0, both would have been ingested as "resources" and polluted the table with entries that are not documents. Contacts (Primary POC, Data Owner) are likewise modeled as people, separate from document resources.

Third, sparsity is a design requirement, not an edge case. With only about 5.5% of views carrying any resource signal, every resource-related column must be nullable and empty-tolerant, and consumers of the DSI tables should treat an empty resource set as the normal state rather than an error.